

Competitive Programming

Section 4 Editorial: Prefix sums and difference arrays

Competitive Programming Course

2026-07-04

Table of contents

1	How to read this editorial	3
2	Problem A: Range Sum Queries	4
2.1	Restatement	4
2.2	Important constraints and traps	4
2.3	Observations	4
2.4	Sample walk-through	4
2.5	Algorithm	5
2.6	Correctness	5
2.7	Complexity	5
2.8	C++ solution	5
2.9	Python solution	6
2.10	Common wrong answers	7
3	Problem B: Range Add Final Array	8
3.1	Restatement	8
3.2	Important constraints and traps	8
3.3	Observations	8
3.4	Sample walk-through	8
3.5	Algorithm	9
3.6	Correctness	9
3.7	Complexity	9
3.8	C++ solution	9
3.9	Python solution	10
3.10	Common wrong answers	11
4	Problem C: Subarray Sum Count	12
4.1	Restatement	12
4.2	Important constraints and traps	12
4.3	Observations	12
4.4	Sample walk-through	12
4.5	Algorithm	13
4.6	Correctness	13
4.7	Complexity	13
4.8	C++ solution	14
4.9	Python solution	14
4.10	Common wrong answers	15
5	Stress testing strategy	16
6	What to carry forward	17

1 How to read this editorial

Prefix sums are simple enough to memorize, but the goal is to understand the interval convention. Most wrong answers in this section come from using the right idea with one endpoint shifted by one.

The three problems form a progression:

1. range queries on a fixed array;
2. range updates on an initially zero array;
3. subarray counting by treating prefix sums as values in a frequency table.

2 Problem A: Range Sum Queries

2.1 Restatement

For each query $[l, r]$, where l and r are 1-indexed inclusive endpoints, print:

```
a_l + a_{l+1} + ... + a_r
```

The array does not change.

2.2 Important constraints and traps

The total $n + q$ is at most 200000, so scanning the queried range every time can be too slow. Values may be large enough that range sums need 64-bit integers.

The main trap is the formula. For 1-indexed inclusive queries, use:

```
prefix[r] - prefix[l - 1]
```

not `prefix[r] - prefix[l]`.

2.3 Observations

Define:

```
prefix[0] = 0  
prefix[i] = a1 + a2 + ... + ai
```

Then `prefix[r]` contains everything up through r , and `prefix[l - 1]` contains everything before l . Their difference is exactly the query range.

2.4 Sample walk-through

Array:

```
1 2 3 4 5
```

Prefix:

i	prefix[i]
0	0
1	1
2	3
3	6
4	10
5	15

Query [2, 4]:

```
prefix[4] - prefix[1] = 10 - 1 = 9
```

which equals $2 + 3 + 4$.

2.5 Algorithm

```
build prefix array of length n + 1
for each query l, r:
    print prefix[r] - prefix[l - 1]
```

2.6 Correctness

By construction, `prefix[i]` is the sum of the first i elements. For query $[l, r]$, `prefix[r]` is:

$$a_1 + a_2 + \dots + a_{l-1} + a_l + \dots + a_r.$$

`prefix[l - 1]` is:

$$a_1 + a_2 + \dots + a_{l-1}.$$

Subtracting removes exactly the elements before l , leaving $a_l + \dots + a_r$. Therefore each printed sum is correct.

2.7 Complexity

Building the prefix array is $O(n)$. Each query is answered in $O(1)$. Memory is $O(n)$.

2.8 C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, q;
        cin >> n >> q;
        vector<long long> prefix(n + 1, 0);
        for (int i = 1; i <= n; ++i) {
            long long x;
            cin >> x;
            prefix[i] = prefix[i - 1] + x;
        }
        while (q--) {
            int l, r;
            cin >> l >> r;
            cout << prefix[r] - prefix[l - 1] << '\n';
        }
    }
    return 0;
}

```

2.9 Python solution

```

import sys

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    q = data[idx + 1]
    idx += 2
    prefix = [0]
    for value in data[idx:idx + n]:
        prefix.append(prefix[-1] + value)
    idx += n
    for _ in range(q):
        l = data[idx]
        r = data[idx + 1]
        idx += 2

```

```
out.append(str(prefix[r] - prefix[l - 1]))
sys.stdout.write("\n".join(out))
```

2.10 Common wrong answers

- Using `prefix[r] - prefix[l]`.
- Building a prefix array without the initial zero.
- Using 32-bit `int` for sums.
- Treating input queries as 0-indexed.

3 Problem B: Range Add Final Array

3.1 Restatement

Start with n zeros. Each update $[l, r, x]$ adds x to every position from l through r , using 1-indexed inclusive endpoints. After all updates, print the final array.

3.2 Important constraints and traps

The total $n + q$ is at most 200000. Applying each update directly to every position could be $O(nq)$.

The difference-array update is:

```
diff[l] += x
diff[r + 1] -= x
```

This requires `diff` to have a safe cell at $n + 1$.

3.3 Observations

Instead of storing values directly, store where changes begin and end. During the final left-to-right scan, the running sum tells us the current value.

An update contributes to position i exactly when:

```
l <= i <= r
```

Adding at l starts the contribution. Subtracting at $r + 1$ stops it.

3.4 Sample walk-through

Updates:

```
[1, 3] += 10
[2, 5] += 1
[4, 4] += -5
```

Difference marks:

```
diff[1] += 10
diff[4] -= 10
diff[2] += 1
diff[6] -= 1
diff[4] += -5
diff[5] -= -5
```

Scanning produces:

```
10 11 11 -4 1
```

3.5 Algorithm

```
diff = zeros of length n + 2
for each update l, r, x:
    diff[l] += x
    diff[r + 1] -= x
cur = 0
for i from 1 to n:
    cur += diff[i]
    output cur
```

3.6 Correctness

Consider any update $[l, r, x]$. In the running sum of `diff`, the update's $+x$ is included starting at position l . Its $-x$ is included starting at position $r + 1$. Therefore, for positions i with $l \leq i \leq r$, the running sum includes exactly x from this update. For positions outside the range, it includes zero net contribution from this update.

Because addition is independent across updates, the running sum at each position equals the sum of all updates covering that position. That is exactly the final array value. The algorithm prints these values, so it is correct.

3.7 Complexity

Each update is processed in $O(1)$. Reconstruction is $O(n)$. Memory is $O(n)$.

3.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);

int T;
cin >> T;
while (T--) {
    int n, q;
    cin >> n >> q;
    vector<long long> diff(n + 2, 0);
    while (q--) {
        int l, r;
        long long x;
        cin >> l >> r >> x;
        diff[l] += x;
        diff[r + 1] -= x;
    }
    long long cur = 0;
    for (int i = 1; i <= n; ++i) {
        cur += diff[i];
        if (i > 1) cout << ' ';
        cout << cur;
    }
    cout << '\n';
}
return 0;
}

```

3.9 Python solution

```

import sys

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    q = data[idx + 1]
    idx += 2
    diff = [0] * (n + 2)
    for _ in range(q):
        l = data[idx]
        r = data[idx + 1]
        x = data[idx + 2]
        idx += 3
        diff[l] += x
        diff[r + 1] -= x
    cur = 0

```

```
values = []
for i in range(1, n + 1):
    cur += diff[i]
    values.append(str(cur))
out.append(" ".join(values))
sys.stdout.write("\n".join(out))
```

3.10 Common wrong answers

- Writing `diff[r] -= x` instead of `diff[r + 1] -= x`.
- Allocating only `n + 1` cells and failing when `r = n`.
- Forgetting negative updates.
- Printing the difference array instead of its prefix reconstruction.

4 Problem C: Subarray Sum Count

4.1 Restatement

Count nonempty contiguous subarrays whose sum is exactly k .

4.2 Important constraints and traps

Negative values are allowed. That means two pointers or sliding window is not valid in general. Prefix sums plus a frequency table works with any signs.

The number of matching subarrays may exceed 32-bit range.

4.3 Observations

Let $\text{prefix}[i]$ be the sum of the first i elements. The subarray $l..r$ has sum:

```
prefix[r] - prefix[l - 1]
```

We want this to be k , so:

```
prefix[l - 1] = prefix[r] - k
```

When scanning the array and computing the current prefix $\text{prefix}[r]$, every previous prefix equal to $\text{prefix}[r] - k$ gives one subarray ending at r .

4.4 Sample walk-through

Array:

```
1 2 3 -2 1
```

Target 3.

Trace:

value	prefix	need prefix	previous count	answer
start	0	-	$\text{freq}[0]=1$	0
1	1	-2	0	0
2	3	0	1	1

value	prefix	need prefix	previous count	answer
3	6	3	1	2
-2	4	1	1	3
1	5	2	0	3

The three subarrays are:

```
1 2
3
2 3 -2
```

4.5 Algorithm

```
freq[0] = 1
prefix = 0
answer = 0
for value in array:
    prefix += value
    answer += freq[prefix - k]
    freq[prefix] += 1
print answer
```

4.6 Correctness

Before processing each position, `freq[p]` equals the number of previous prefix sums with value `p`. The current prefix after reading position `r` is `prefix[r]`.

A subarray ending at `r` has sum `k` exactly when its previous prefix is `prefix[r] - k`. The map contains exactly the number of such previous prefixes, so the algorithm adds exactly the number of valid subarrays ending at `r`. Then it inserts the current prefix so it can serve as a previous prefix for future positions.

The initial `freq[0] = 1` represents the empty prefix before the array starts. It is what allows subarrays beginning at position 1 to be counted.

Every nonempty subarray has a unique ending position and is counted when that ending position is processed. Therefore the final answer is correct.

4.7 Complexity

The algorithm does one map lookup and one map update per element. Expected time is $O(n)$, and memory is $O(d)$, where `d` is the number of distinct prefix sums.

4.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        long long target;
        cin >> n >> target;
        unordered_map<long long, long long> freq;
        freq.reserve(2 * n + 1);
        freq[0] = 1;
        long long prefix = 0;
        long long answer = 0;
        for (int i = 0; i < n; ++i) {
            long long x;
            cin >> x;
            prefix += x;
            auto it = freq.find(prefix - target);
            if (it != freq.end()) {
                answer += it->second;
            }
            ++freq[prefix];
        }
        cout << answer << '\n';
    }
    return 0;
}
```

4.9 Python solution

```
from collections import defaultdict
import sys

def count_subarrays(values: list[int], target: int) -> int:
    freq = defaultdict(int)
    freq[0] = 1
    prefix = 0
    answer = 0
    for value in values:
        prefix += value
```

```

        answer += freq[prefix - target]
        freq[prefix] += 1
    return answer

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    target = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    out.append(str(count_subarrays(values, target)))
sys.stdout.write("\n".join(out))

```

4.10 Common wrong answers

- Forgetting `freq[0] = 1`.
- Inserting the current prefix before counting, which can count empty subarrays when `k = 0`.
- Using a sliding window even though negative values are allowed.
- Using `int` for prefix sums or answer in C++.

5 Stress testing strategy

Prefix-sum code is easy to test with a slow oracle.

For range sums, compare each query to a direct sum of the slice.

For range updates, apply each update directly to a small array and compare the final result with the difference-array result.

For subarray sums, use the quadratic loop:

```
answer = 0
for l:
    total = 0
    for r from l to n - 1:
        total += a[r]
        if total == k:
            answer += 1
```

This slow oracle caught incorrect hand-written expected values while this section was being built. That is exactly why stress tests are worth having.

6 What to carry forward

Prefix sums turn range sums into subtraction. Difference arrays turn range updates into boundary marks. Prefix-frequency maps turn subarray conditions into complement lookups.

These ideas reappear in:

- two-dimensional prefix sums;
- Fenwick trees and segment trees;
- offline range queries;
- subarray counting with modular conditions;
- dynamic programming transitions over intervals.